ENS 491-492 – Graduation Project

Final Report

Project Title: . . . . . .

Group Members: . . .

Supervisor(s): . . .

Date:

- **ENS 491-492 Final Report must contain the following sections (leave section titles unchanged).**
- **You are required to write clearly and concisely: The main body of the report (not including the Title Page, Appendix, and References) must be between 20–25 pages.**
- **Project advisor may have additional requirements for the report.**

1. EXECUTIVE SUMMARY

This ongoing research investigates how a large language model can be fine-tuned to classify phishing websites while producing explanations that are grounded in observable webpage evidence. The central problem is not only whether a webpage is phishing or benign, but whether the model can justify its verdict using signals that a security analyst can inspect, such as URL structure, title-domain consistency, forms, credential fields, redirects, external resources, and other HTML-derived artifacts. The intended output is a constrained JSON response containing a verdict, confidence level, evidence items, and a short explanation, rather than an unconstrained natural-language rationale.

The project uses a supporting data pipeline to collect and normalize webpages from phishing feeds, benign website rankings, and live security-scan sources. This infrastructure is necessary because LLM fine-tuning requires consistent inputs and reliable supervision, but it is not the primary contribution. The research focus is the design of a fine-tuning dataset, prompt schema, evidence schema, and evaluation procedure for an explainable phishing classifier. The current experimental direction uses parameter-efficient fine-tuning of a compact multimodal-capable language model in text-only mode, using URL, page title, and engineered URL/HTML/metadata features as model-visible inputs.

Work completed so far includes webpage collection, feature extraction, dataset construction, train-only feature selection, and baseline feature analysis. Classical models and explanation tools such as SHAP, LIME, and EBM are used as diagnostic instruments to understand feature behavior, dataset bias, and evidence candidates; they are not treated as the final research result. The main expected outcome is a fine-tuned LLM that can produce valid, evidence-grounded, schema-compliant phishing classifications. The project is still in progress, and final claims will require evaluation on domain-grouped and time-aware splits, schema-validity metrics, grounded-evidence metrics, and classification metrics on held-out data.

2. PROBLEM STATEMENT

Phishing websites remain a persistent security problem because they exploit user trust and imitate legitimate online services. They frequently collect credentials, payment information, or personal data through pages that look familiar to users but are hosted on unrelated or compromised infrastructure. The Anti-Phishing Working Group reported 853,244

phishing attacks in the fourth quarter of 2025 and 3.8 million attacks during 2025, with Social Media and SaaS/Webmail among the most frequently targeted sectors [1]. These attacks are difficult to handle with static defenses alone because phishing pages are short-lived, change quickly, and often reuse legitimate-looking assets.

The research problem addressed in this project is how to fine-tune an LLM so that it can perform explainable phishing website classification. A conventional classifier can output a phishing/benign label, but such a label is insufficient for analyst triage, reproducibility, and trust. A useful model should produce a structured verdict and cite the evidence that supports it. This creates three connected challenges. First, the model input must be compact enough for fine-tuning while preserving discriminative webpage signals. Second, the target output must force the model to cite grounded evidence instead of producing generic or hallucinated explanations. Third, evaluation must measure both classification quality and explanation quality, because a model can predict the correct label for the wrong reason.

The project therefore treats webpage collection and feature extraction as enabling infrastructure for LLM fine-tuning. Raw webpages are normalized into URL, title, redirect, HTML, and metadata observations. Deterministic features are extracted from these observations, including URL structure, visible-text statistics, title-domain overlap, form and input counts, password and email fields, internal/external link structure, script and resource ratios, hidden elements, JavaScript indicators, and redirect behavior. These features are used to construct prompts and evidence-grounded targets for a fine-tuned model. Source and label metadata are kept for diagnostics, but are excluded from model-visible prompts to reduce leakage.

The current fine-tuning workflow uses a domain-grouped split before feature selection, computes feature selection only on the training split, and trains a LoRA adapter for strict JSON phishing verdicts. The model is instructed to use only the URL, title, and provided engineered features, and to return a JSON object with 'verdict', 'confidence_level', three to seven evidence items, and a short explanation. Post-training evaluation is designed to measure parseability, schema validity, evidence-count compliance, whether cited evidence features are actually present in the prompt, duplicate evidence, and classification performance on validation and test splits. Classical Random Forest models, SHAP, LIME, and EBM analyses are used during development to inspect feature behavior and dataset bias, but the main research objective remains LLM fine-tuning for grounded explanations.

Previous work. Hannousse and Yahiouche proposed a reproducible benchmark construction scheme for website phishing detection and built an 11,430-sample dataset with 87 URL, content, and external-service features [7]. Their work motivates reproducible feature extraction and benchmarking, but their main framing is classical website phishing detection rather than fine-tuned structured-output LLM classification.

Aljofey et al. combined URL and HTML information using URL character-sequence features, TF-IDF features from webpage text and HTML attributes, hyperlink features, and login-form features [8]. Their results support the assumption that URL-only models miss important webpage evidence. This project adopts the same multimodal webpage

view, but converts the evidence into an LLM-readable structured representation.

Guo et al. introduced HinPhish, which models webpages as a heterogeneous information network with domain and resource nodes and link relations such as local, relative, and outlier links [9]. This is relevant because it treats webpage relations as important phishing evidence. The present project uses simpler engineered relation features so they can be cited directly in JSON evidence.

Lin et al. proposed Phishpedia, a visual phishing identification approach that detects logos in screenshots, matches logo variants, and reports the inferred target brand [10]. This work is important because it emphasizes explainable phishing identification, not only binary detection. The present project currently focuses on text and engineered features, but shares the goal of producing analyst-readable evidence.

Opara et al. proposed WebPhish, an end-to-end neural model trained on raw URL and HTML character embeddings, reducing reliance on manually selected features [11]. In contrast, this project deliberately keeps engineered evidence visible to the model because the goal is not only accuracy but also faithful explanation.

Trad and Chehab compared prompt engineering and fine-tuning for phishing URL detection with large language models, reporting the practical importance of fine-tuning for stable phishing classification behavior [12]. This directly motivates the project's decision to move beyond prompt-only experiments and train a domain-adapted model.

Recent LLM-based phishing systems further motivate the research direction. PhishLLM uses LLM knowledge and validation to infer brand-domain relationships and credential-taking intent without a predefined reference list [13]. KnowPhish combines LLM-based brand extraction with multimodal knowledge graphs for reference-based phishing detection [14]. PhishLang demonstrates that lightweight language models can be useful for client-side phishing detection under latency and memory constraints [15]. These works support the broader hypothesis that language models can contribute to phishing detection when inputs, supervision, and evaluation are carefully constrained.

2.1. Objectives/Tasks

1. Construct a research dataset for explainable phishing classification from collected phishing, benign, and live-scan webpages.
2. Normalize webpages into compact model inputs containing URL, final URL, title, and selected engineered URL/HTML/metadata features.
3. Design a strict JSON target schema containing verdict, confidence level, evidence items, and a short evidence-grounded explanation.
4. Fine-tune a language model with LoRA/QLoRA-style parameter-efficient adaptation for the structured phishing classification task.
5. Use classical models, SHAP, LIME, and EBM as supporting analyses to identify feature behavior, candidate evidence, and dataset-bias risks.

6. Evaluate both prediction and explanation quality using classification metrics, schema-validity metrics, grounded-evidence metrics, and domain-aware held-out splits.

2.2. Realistic Constraints

The project is constrained by data availability, compute cost, ethical handling of malicious webpages, and the difficulty of evaluating explanations. Phishing pages are transient and adversarial, so collection must preserve page state while avoiding unsafe interaction with live malicious content. Model-visible prompts must avoid hidden labels, collection-source identifiers, raw crawler errors, timestamps, API keys, and potentially sensitive URL parameters. External reputation and scan services are useful for collection and diagnostics, but their outputs can be delayed, rate-limited, noisy, or unavailable; therefore they should not become opaque shortcuts for the fine-tuned model.

Fine-tuning also introduces practical constraints. Full-parameter training of a large model is expensive, so the project uses parameter-efficient adaptation inspired by LoRA and QLoRA [16, 17]. Long HTML documents cannot be passed directly to the model without exceeding token budgets, so inputs must be reduced to selected text and engineered features. Finally, explanation quality cannot be judged by readability alone. The evaluation must check whether generated evidence cites features actually present in the prompt, whether the evidence supports the predicted verdict, and whether performance holds under domain-grouped and time-aware splits. Current feature-analysis results are therefore treated as development evidence, not as final claims of completed system performance.

3. METHODOLOGY

3.1. Data Collection

The data collection stage was designed to build a reproducible raw webpage corpus for later explainable phishing classification. At this stage, the objective is not to engineer model features, train the classifier, or evaluate explanations. The objective is to acquire webpages from complementary sources, preserve the observable page state, record collection metadata, and store each observation in a consistent database format. Later stages use this raw collection snapshot as input for feature engineering and fine-tuning.

Each collected item represents one attempted webpage observation. The stored record keeps the original requested URL, the final URL after redirects when available, the rendered page title, raw HTML, crawl timestamp, HTTP status, response headers, redirect history, content length, elapsed request time, and collection errors when the page cannot be retrieved. Domain registration information is collected through RDAP where possible. For live security observations, external scan and reputation results are stored separately from browser-crawl content so that later modeling can decide which fields are safe to expose to the model.

3.1..1 Collection Sources

Phishing feed collection. The phishing source is the OpenPhish academic CSV feed [3]. A scheduled collector polls the feed from GitHub, optionally using a GitHub personal access token for authenticated access. The collector runs periodically, with the default update interval set to ten minutes. Each feed row is treated as a candidate phishing URL and inserted into the MongoDB collection `phishing_db.phishing_urls`. A unique index on the `url` field prevents duplicate insertion across polling cycles. New URLs are stored with an `added_at` timestamp, while already known URLs are skipped.

A separate browser crawler consumes unvisited URLs from the phishing URL queue. Before crawling, records without a visit marker are initialized as unvisited. The crawler then places unvisited URLs into a worker queue and uses multiple parallel workers; the implementation default is four workers. Each worker opens the URL with a headless browser session configured for stealthier rendering, network-idle waiting, DOM loading, Cloudflare-solving support, a 60-second timeout, and a five-second post-load wait. For each successful fetch, the crawler extracts the page title, raw HTML, response status, headers, encoding, final URL, redirect count, redirect history, elapsed time, content length, and crawl timestamp. The crawler also performs RDAP lookup for the registered domain or IP address. The resulting page record is stored in `phishing_db.website_content`. Whether the crawl succeeds or fails, the source URL is marked as visited and receives a `visited_at` timestamp; failures retain an error field for auditability.

Benign ranked-domain collection. Benign collection uses the Tranco ranked-domain list [2]. The local collector reads a Tranco CSV file, normalizes each domain by lowercasing it, extracting the hostname, removing a leading `www.`, and removing a trailing dot. The collector stores records in `tranco.websites` and maintains a unique index on `url`. Before each batch, it reads already fetched domains from stored URLs and requested-domain metadata, then skips domains that have already been collected.

The Tranco crawler processes domains in batches, with defaults of ten domains per batch and four concurrent workers. For each apex domain, the crawler tries `https://www.<domain>` before `https://<domain>`; non-apex domains are attempted directly. Before launching the browser, the crawler performs an HTTP preflight request using a browser-like user agent, English `Accept-Language` headers, redirects enabled, and a five-second timeout. SSL failures are retried with certificate verification disabled for the preflight only. Responses with 4xx or 5xx status codes are normally skipped, except for 403 responses, because a browser session may still pass bot-protection pages that block simple HTTP clients.

If the preflight is acceptable, the crawler opens the candidate URL in a headless browser session with network-idle waiting, DOM loading, optional Cloudflare solving, pinned locale and timezone settings, and request timeouts. The first candidate that returns a usable response below HTTP 400 is stored. The saved document follows the same general shape as the phishing crawl: requested URL, final URL, title, HTML, re-

sponse metadata, redirect information, RDAP data, timestamp, and any error field. This symmetry is important because it prevents the benign and phishing sources from having incompatible raw schemas.

Live scan and reputation collection. The third collection stream records live security observations and delayed rescans. This component monitors the `urlscan.live` collection and schedules external checks through VirusTotal, Google Safe Browsing, and urlscan-compatible live observations [4, 5, 6]. It creates indexes for URL lookup, pending scan state, delayed rescan completion, and significant-change detection. The scan workflow separates an initial observation, denoted T0, from a delayed observation, denoted T7. The default T7 delay is seven days, which allows the project to record whether a suspicious page, redirect target, or reputation result changes after the initial capture.

Because external services impose limits, the scanner uses a key pool and sliding-window rate limiter for VirusTotal requests. The implementation defaults to three VirusTotal submissions per 75 seconds, 480 submissions per day per key, a minimum spacing of 17 seconds, and a cooldown period for exhausted or failed keys. Pending analyses are polled after a settle interval rather than assumed to be immediately complete. Google Safe Browsing checks are made through the `threatMatches:find` endpoint when an API key is available. For significant T7 changes, the scanner can schedule a follow-up scrape while limiting captured HTML size to 524,288 bytes. This stream is therefore used to preserve temporal and reputation evidence, while still maintaining explicit state for pending, completed, failed, and delayed scans.

3.1..2 Collection Snapshot

The current raw collection snapshot used by the project contains 147,524 webpage records. The snapshot combines phishing-feed crawls, live scan observations, and Tranco benign crawls. Table 1 summarizes the source composition. The labels are not collected from one single service: phishing-feed crawls are treated as malicious candidates, Tranco crawls are treated as benign candidates, and live scan records are labeled from their collection and reputation context during dataset construction. The resulting label distribution in this snapshot is 87,924 benign records and 59,600 phishing records.

Table 1: Raw collection snapshot by source.

Stored source	Records	Collection role
`phishing_db.website_content`	50,000	Feed-derived phishing webpage crawls
`urlscan.live`	49,336	Live scan and reputation observations
`tranco.websites`	48,188	Ranked-domain benign webpage crawls
Total	**147,524**	**Raw collection snapshot**

The collection snapshot also contains a measurable source-label dependency. This is expected because the three acquisition channels were chosen for different purposes: phishing feeds provide malicious candidates, Tranco provides benign candidates, and

live scan observations provide mixed security context. The source identifier is therefore retained for auditing and bias analysis, but it is treated as collection metadata rather than as a model-visible signal in later stages.

3.1..3 Collection Graphs

Figure 1 shows the data collection pipeline from external sources to the raw collection snapshot. The figure intentionally stops before feature engineering, because feature extraction, feature selection, and prompt construction are separate methodology stages.

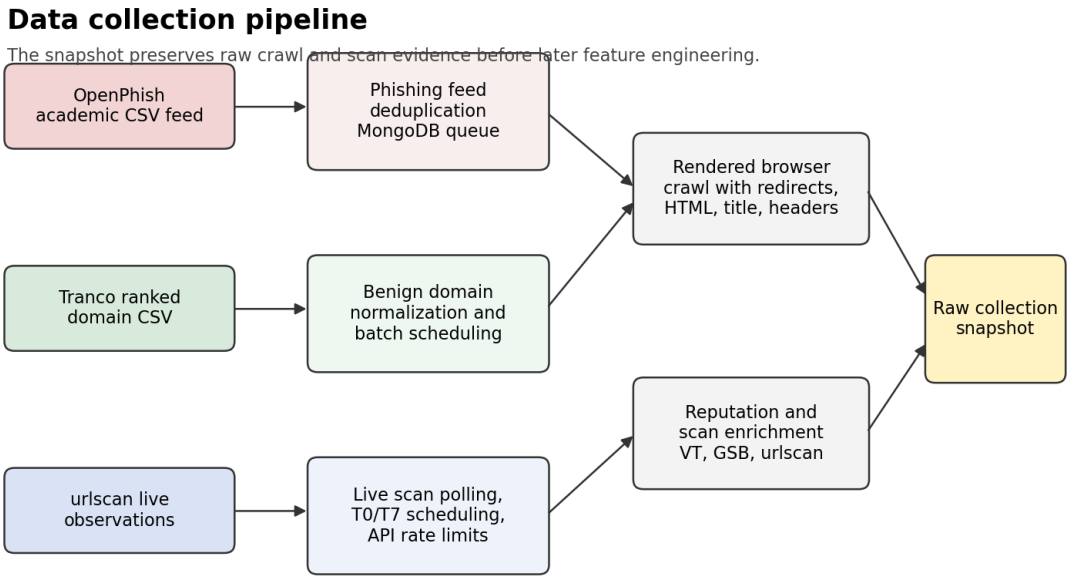

Figure 1: Data collection pipeline before feature engineering.

Figures 2 and 3 summarize the current collection snapshot. The source-count graph is useful for checking whether one acquisition channel dominates the raw corpus. The label-count graph shows the class balance that later stages must account for during splitting and evaluation.

3.1..4 Collection Algorithms

The collection procedures are summarized below as numbered pseudocode. The algorithms describe collection behavior only; feature engineering and model-input construction are intentionally left to the following methodology stage.

4. RESULTS & DISCUSSION

Give detailed description of the results/achievements including the information:

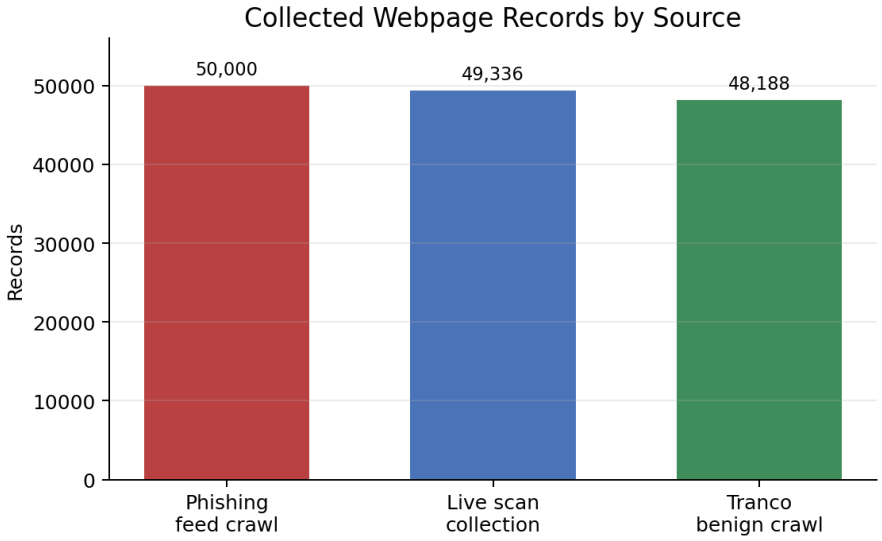

Figure 2: Record counts by collection source.

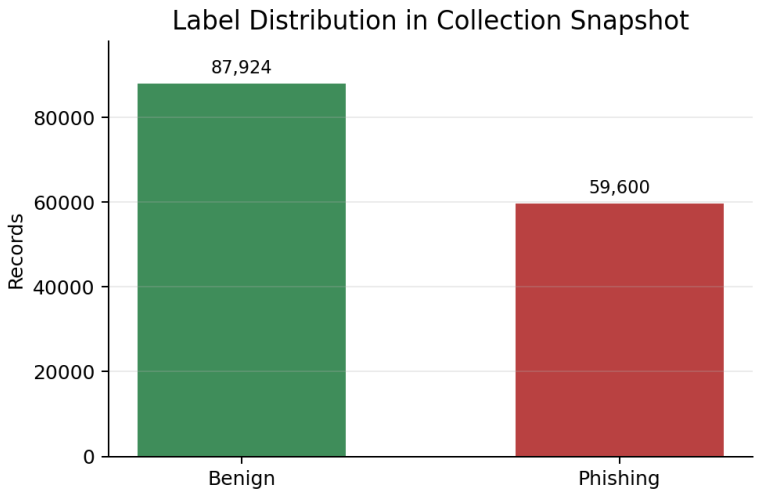

Figure 3: Label distribution in the current collection snapshot.

Algorithm 1 Feed-based phishing webpage collection

Require: OpenPhish academic CSV feed, MongoDB connection, polling interval
Ensure: Raw phishing webpage records in `phishing_db.website_content`

Poll the OpenPhish academic CSV feed at the configured interval.
for all rows in the feed **do**
    Parse the row as a candidate phishing URL.
    if the URL is not already in the phishing URL queue **then**
        Insert the URL with an `added_at` timestamp.
    end if
end for
Select URLs that are not marked as visited.
Enqueue selected URLs for bounded parallel crawling.
for all queued URLs **do**
    Open the URL in a headless rendered-browser session.
    Wait for DOM loading, network idle, and the configured post-load delay.
    Record title, HTML, final URL, status, headers, redirects, timing, and size.
    Add RDAP data for the registered domain or IP address.
    Store the crawl result, or the crawl error, in the webpage collection.
    Mark the source URL as visited with a `visited_at` timestamp.
end for

- *To what extent the initial objectives were realized?*
- *How they differed from the initial objectives?*
- *Is this project successfully completed?*
- *Did the project have meaningful contribution to previous state-of-art?*
- *What is the estimated carbon footprint of the proposed solution (e.g., kg CO_2 eq per functional unit) and how does it compare with conventional alternatives?*
- *How does the proposed solution perform in terms of economic feasibility when considering cost, supply chain efficiency, and circularity aspects (e.g., reuse, recycling, resource efficiency)?*

5. IMPACT

Describe the scientific, technological or socio-economic impacts of your achievements.

- *Highlight (if any) innovative and commercial/entrepreneurial aspects of the project.*
- *Is there any Freedom-to-Use (FTU) issues?*
- *What are the environmental and sustainability impacts of the proposed solution (e.g., carbon footprint reduction, energy efficiency, resource utilization)?*

Algorithm 2 Tranco benign webpage collection

Require: Tranco CSV file, MongoDB connection, batch size, concurrency limit
Ensure: Raw benign webpage records in `tranco.websites`

Read ranked domains from the Tranco CSV file.
for all domains in rank order **do**
    Normalize the domain by hostname extraction, lowercasing, and `www.` removal.
    if the normalized domain has already been collected **then**
        Skip the domain.
    else
        Add the domain to the next crawl batch.
    end if
end for
for all domains in the batch, using bounded concurrency **do**
    Build HTTPS candidates, trying the `www` form first for apex domains.
    for all candidate URLs **do**
        Run HTTP preflight with browser-like headers and redirects enabled.
        if preflight status is a non-403 client or server error **then**
            Try the next candidate URL.
        else
            Open the candidate in a headless rendered-browser session.
            if the browser response status is below HTTP 400 **then**
                Store title, HTML, final URL, response metadata, redirects, RDAP data, and timestamp.
                Stop processing candidates for this domain.
            end if
        end if
    end for
end for

Algorithm 3 Live scan and delayed rescan collection

Require: Live URL observations, VirusTotal keys, optional Safe Browsing and urlscan keys

Ensure: T0 and T7 scan records with pending, completed, failed, and change states

Maintain indexes for URL lookup, pending analyses, T7 scheduling, and change flags.
while the scanner is running **do**
    if a T7 scrape is pending after significant change **then**
        Perform a bounded follow-up scrape and save the result.
    else if an external analysis is pending and ready to poll **then**
        Poll the provider and update the pending scan state.
    else if a URL is missing T0 or is due for T7 **then**
        if the relevant API rate limiter permits a request **then**
            Submit the scan request and store the pending analysis identifier.
        end if
    else
        Backfill from live observations until new scan work is available.
    end if
    Compare delayed T7 observations with T0 and mark significant changes.
end while

Algorithm 4 Raw collection snapshot construction

Require: Phishing crawl collection, Tranco crawl collection, live scan collection

Ensure: Auditable raw snapshot for the next methodology stage

Read collected records from the phishing, benign, and live-scan collections.
Preserve collection source, crawl metadata, scan metadata, and label provenance.
Remove exact duplicate observations for the same stored URL and collection state.
Count records by source and label for collection auditing.
Produce the snapshot totals reported in Table 1.
Pass the raw snapshot to the next methodology stage.

- *How does the solution contribute to circularity and sustainable engineering practices?*

6. ETHICAL ISSUES

In this section, you are required to identify (if any) ethical aspects of your project/solution. For instance, are there possible legal/ethical consequences of the solution? Some examples are given below:

- *products implicitly using patent protected designs/concepts;*
- *materials that have better appearance but are toxic*
- *under design for profit*
- *products for secret survey of personal private life*
- *solutions that may have negative affect on health)?*

If there aren't any, you need to still keep this section and state that there are no ethical issues.

7. PROJECT MANAGEMENT

Initial and final project plans. How, during implementation, project plan has changed and why. What have you learnt in terms of managing your project?

8. CONCLUSION AND FUTURE WORK

Briefly discuss the most important results/findings of your project. Point out any limitations for your study and conclusions. What are the logical next steps of your project? Provide a short discussion on where the project could go next, if you or other students continued the work. How can the solution be further improved in terms of sustainability, cost efficiency, and real-world implementation?

9. APPENDIX

Provide, if necessary, additional material such as source code, CAD drawings, proofs of results, etc.

10. REFERENCES

[1] Anti-Phishing Working Group. *Phishing Activity Trends Report, 4th Quarter 2025*. Released December 10, 2025. https://docs.apwg.org/reports/apwg_trends_report_q4_2025.pdf

[2] V. Le Pochat, T. Van Goethem, S. Tajalizadehkhoob, M. Korczynski, and W. Joosen. "Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation." NDSS, 2019. https://tranco-list.eu/

[3] OpenPhish. "OpenPhish Academic Feed." https://github.com/openphish/academic

[4] VirusTotal. "Analyses Object." VirusTotal API Documentation. https://docs.virustotal.com/reference/analyses-object

[5] urlscan.io. "API Documentation." https://urlscan.io/docs/api/

[6] Google. "Safe Browsing API v4." https://developers.google.com/safe-browsing/v4

[7] A. Hannousse and S. Yahiouche. "Towards Benchmark Datasets for Machine Learning Based Website Phishing Detection: An Experimental Study." *Engineering Applications of Artificial Intelligence*, 104:104347, 2021. https://doi.org/10.1016/j.engappai.2021.104347

[8] A. Aljofey, Q. Jiang, A. Rasool, H. Chen, W. Liu, Q. Qu, and Y. Wang. "An Effective Detection Approach for Phishing Websites Using URL and HTML Features." *Scientific Reports*, 12:8842, 2022. https://doi.org/10.1038/s41598-022-10841-5

[9] B. Guo, Y. Zhang, C. Xu, F. Shi, Y. Li, and M. Zhang. "HinPhish: An Effective Phishing Detection Approach Based on Heterogeneous Information Networks." *Applied Sciences*, 11(20):9733, 2021. https://doi.org/10.3390/app11209733

[10] Y. Lin, R. Liu, D. M. Divakaran, J. Y. Ng, Q. Z. Chan, Y. Lu, Y. Si, F. Zhang, and J. S. Dong. "Phishpedia: A Hybrid Deep Learning Based Approach to Visually Identify Phishing Webpages." USENIX Security Symposium, 2021. https://www.usenix.org/conference/usenixsecurity21/presentation/lin

[11] C. Opara, B. Wei, and Y. Chen. "Look Before You Leap: Detecting Phishing Web Pages by Exploiting Raw URL and HTML Characteristics." arXiv:2011.04412, 2020. https://arxiv.org/abs/2011.04412

[12] F. Trad and A. Chehab. "Prompt Engineering or Fine-Tuning? A Case Study on Phishing Detection with Large Language Models." *Machine Learning and Knowledge Extraction*, 6(1), 2024. https://www.mdpi.com/2504-4990/6/1/18

[13] R. Liu et al. "Less Defined Knowledge and More True Alarms: Reference-based Phishing Detection without a Pre-defined Reference List." USENIX Secu-

rity Symposium, 2024. https://www.usenix.org/conference/usenixsecurity24/presentation/liu-ruofan

[14] Y. Li et al. “KnowPhish: Large Language Models Meet Multimodal Knowledge Graphs for Enhancing Reference-Based Phishing Detection.” USENIX Security Symposium, 2024. https://www.usenix.org/conference/usenixsecurity24/presentation/li-yuexin

[15] S. S. Roy and S. Nilizadeh. “PhishLang: A Lightweight, Client-Side Phishing Detection Framework using MobileBERT for Real-Time, Explainable Threat Mitigation.” arXiv:2408.05667, 2024. https://arxiv.org/abs/2408.05667

[16] E. J. Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models.” arXiv:2106.09685, 2021. https://arxiv.org/abs/2106.09685

[17] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. “QLoRA: Efficient Finetuning of Quantized LLMs.” NeurIPS, 2023. https://arxiv.org/abs/2305.14314